

Time and Space Complexity – Interview Handbook

This handbook summarizes how to analyze algorithm complexity in a professional software engineering or quant developer interview.

1. What is Big-O?

Big-O describes how runtime or memory grows as input size n increases. Ignore constants and lower-order terms because asymptotic growth dominates.

2. Six-Step Time Complexity Analysis

- 1) Count loops.
- 2) Check how loop variables change (e.g. divide by 2 $\rightarrow O(\log n)$).
- 3) Independent loops add: $O(n)+O(n)=O(n)$.
- 4) Nested loops multiply: $O(n*m)$.
- 5) Analyze recursion (recursion tree / Master Theorem).
- 6) Include built-in operations such as dict lookup, sort, heap operations.

3. Space Complexity

Count extra memory only: variables $O(1)$, output arrays $O(n)$, hash maps $O(n)$, recursion stack $O(\text{depth})$, BFS queue $O(\text{width})$.

4. Python Data Structure Cheat Sheet

`list.append` $O(1)$, `list.pop()` $O(1)$, `list.pop(0)` $O(n)$, `deque.append/popleft` $O(1)$, dict/set lookup average $O(1)$, sort $O(n \log n)$, heap push/pop $O(\log n)$, `bisect` $O(\log n)$.

5. Common Algorithms

Binary Search: $O(\log n)/O(1)$
Merge Sort: $O(n \log n)/O(n)$
Quick Sort: Avg $O(n \log n)$, Worst $O(n^2)$, Space $O(\log n)$
DFS: $O(V+E)$, recursion stack $O(V)$
BFS: $O(V+E)$, queue $O(V)$
Sliding Window: $O(n)$
Two Pointers: $O(n)$.

6. Interview Answer Template

'The algorithm scans the array once, so each element is processed exactly one time. Hash map lookup is $O(1)$ on average. Therefore the overall time complexity is $O(n)$. The hash map stores at most n elements, so the space complexity is $O(n)$.'

7. Common Pitfalls

`list.remove()` is $O(n)$; '`x in list`' is $O(n)$ but '`x in set`' is $O(1)$ average; `".join(strings)` is preferred over repeated string concatenation.

8. Practice Problems

Two Sum, Merge Intervals, Top K Frequent Elements, LRU Cache, Binary Search, Number of Islands, Word Break, Merge K Lists.